

EPIC 04 — Task 02: Implement JWT Authentication

Your task requires you to create:

- Login API
- User registration
- Password hashing
- JWT generation
- JWT validation

Step 1 — Add dependencies

Open your **pom.xml** and add these dependencies inside `<dependencies>`:

```
<!-- Spring Security -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

```
<!-- JWT -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
```

```
<artifactId>jjwt-jackson</artifactId>  
<version>0.12.6</version>  
<scope>runtime</scope>  
</dependency>
```

Save pom.xml.

Wait for Maven to download the dependencies.

Step 2 — Create the User model

Create package:

```
com.example.userservice.model
```

Create:

```
User.java
```

```
package com.example.userservice.model;
```

```
public class User {
```

```
    private Long id;
```

```
    private String username;
```

```
    private String password;
```

```
    public User() {
```

```
    }
```

```
    public User(Long id, String username, String password) {
```

```
        this.id = id;
```

```
        this.username = username;
```

```
        this.password = password;
```

```
    }
```

```
public Long getId() {  
    return id;  
}  
  
public void setId(Long id) {  
    this.id = id;  
}  
  
public String getUsername() {  
    return username;  
}  
  
public void setUsername(String username) {  
    this.username = username;  
}  
  
public String getPassword() {  
    return password;  
}  
  
public void setPassword(String password) {  
    this.password = password;  
}  
}
```

Step 3 — Create Login Request

Create:

LoginRequest.java

```
package com.example.userservice.model;
```

```
public class LoginRequest {
```

```
    private String username;
```

```
    private String password;
```

```
public String getUsername() {  
    return username;  
}  
  
public void setUsername(String username) {  
    this.username = username;  
}  
  
public String getPassword() {  
    return password;  
}  
  
public void setPassword(String password) {  
    this.password = password;  
}  
}
```

Step 4 — Create Register Request

Create:

RegisterRequest.java

```
package com.example.userservice.model;  
  
public class RegisterRequest {  
  
    private String username;  
    private String password;  
  
    public String getUsername() {  
        return username;  
    }  
  
    public void setUsername(String username) {  
        this.username = username;  
    }  
}
```

```
public String getPassword() {  
    return password;  
}  
  
public void setPassword(String password) {  
    this.password = password;  
}  
}
```

Step 5 — Create JWT Service

Create package:

com.example.userservice.security

Create:

JwtService.java

```
package com.example.userservice.security;  
  
import io.jsonwebtoken.Jwts; import io.jsonwebtoken.security.Keys; import  
org.springframework.stereotype.Service;  
  
import javax.crypto.SecretKey; import java.util.Date;  
  
@Service public class JwtService {  
  
    private final String SECRET =  
        "mySecretKeyForJwtAuthentication12345678901234567890";  
  
    private final SecretKey key =  
        Keys.hmacShaKeyFor(SECRET.getBytes());  
  
    public String generateToken(String username) {  
  
        return Jwts.builder()  
            .subject(username)  
            .issuedAt(new Date())
```

```
        .expiration(
            new Date(System.currentTimeMillis() + 1000 * 60 * 30)
        )
        .signWith(key)
        .compact();
    }
```

```
public String extractUsername(String token) {
```

```
    return Jwts.parser()
        .verifyWith(key)
        .build()
        .parseSignedClaims(token)
        .getPayload()
        .getSubject();
}
```

```
public boolean validateToken(String token, String username) {
```

```
    try {
        String tokenUsername = extractUsername(token);

        return tokenUsername.equals(username);

    } catch (Exception e) {
        return false;
    }
}
```

This creates a JWT that expires after **30 minutes**.

Step 6 — Create Authentication Controller

Create:

AuthController.java

```
package com.example.userservice.controller;

import com.example.userservice.model.LoginRequest;
import com.example.userservice.model.RegisterRequest;
import com.example.userservice.model.User;
import com.example.userservice.security.JwtService;

import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final PasswordEncoder passwordEncoder;
    private final JwtService jwtService;

    private User registeredUser;

    public AuthController(
        PasswordEncoder passwordEncoder,
        JwtService jwtService) {

        this.passwordEncoder = passwordEncoder;
        this.jwtService = jwtService;
    }

    @PostMapping("/register")
    public String register(
        @RequestBody RegisterRequest request) {

        String hashedPassword =
            passwordEncoder.encode(request.getPassword());

        registeredUser = new User(
            1L,
```

```

        request.getUsername(),
        hashedPassword
    );

    return "User registered successfully";
}

@PostMapping("/login")
public String login(
    @RequestBody LoginRequest request) {

    if (registeredUser == null) {
        return "User not registered";
    }

    if (!registeredUser.getUsername()
        .equals(request.getUsername())) {

        return "Invalid username";
    }

    if (!passwordEncoder.matches(
        request.getPassword(),
        registeredUser.getPassword())) {

        return "Invalid password";
    }

    return jwtService.generateToken(
        registeredUser.getUsername()
    );
}
}

```


Step 7 — Configure Password Hashing

Create:

SecurityConfig.java

inside:

com.example.userservice.config

Use:

```
package com.example.userservice.config;
```

```
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
import org.springframework.security.config.annotation.web.builders.HttpSecurity;  
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;  
import org.springframework.security.crypto.password.PasswordEncoder;  
import org.springframework.security.web.SecurityFilterChain;
```

```
@Configuration
```

```
public class SecurityConfig {
```

```
    @Bean
```

```
    public PasswordEncoder passwordEncoder() {  
        return new BCryptPasswordEncoder();  
    }
```

```
    @Bean
```

```
    public SecurityFilterChain securityFilterChain(  
        HttpSecurity http) throws Exception {
```

```
        http
```

```
            .csrf(csrf -> csrf.disable())
```

```
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(
                "/api/auth/register",
                "/api/auth/login"
            ).permitAll()
            .anyRequest().authenticated()
        );

        return http.build();
    }
}
```

Now /register and /login are accessible without authentication.

Step 8 — Test Registration

Start your application.

In Postman:

POST

<http://localhost:8080/api/auth/register>

Select:

Body → raw → JSON

Send:

```
{
  "username": "raju",
  "password": "12345"
}
```

Response:

User registered successfully

The password is stored as a **BCrypt hash**, not as plain text.

Step 9 — Test Login

Send:

POST

<http://localhost:8080/api/auth/login>

Body:

```
{  
  "username": "raju",  
  "password": "12345"  
}
```

You should receive a JWT similar to:

eyJhbGciOiJIUzI1NiJ9...

Copy this token.

Step 10 — JWT validation

For a protected API, the client sends:

Authorization: Bearer <JWT>

The server checks:

JWT received



Signature validation



Token expiration check



Username extraction



Allow / Reject request

If the token is invalid or expired, the request should be rejected.

Requirement	Implementation
Login API	/api/auth/login
User registration	/api/auth/register
Password hashing	BCryptPasswordEncoder
JWT generation	JwtService
JWT validation	validateToken()
Authentication	Spring Security

APIs to test

POST /api/auth/register

POST /api/auth/login